

Part 1: The speed of system calls vs. library calls

- Task 1:

I wrote an execution file called [run.sh](#) in the part1 folder.

```
#!/bin/bash

rm -f result.txt

for i in {1..1000}
do
    ./prog | grep -E "f1|f2" | tr -d '\n' >> result.txt
    echo "" >> result.txt
done

sort result.txt | uniq -c
```

It basically runs the prog 1000 times, with each time it keeps the result in f1f2 or f2f1 and prints it in result.txt, and at last counts the occurrence count.

- Task2:

```
1000 f1f2
0 f2f1
```

is the result after I ran the [run.sh](#) program. It shows that f1 always runs before f2, without exception.

- Task3:

This behavior doesn't guarantee that happen all the time; there might be some cases where f2 runs before f1. Why?

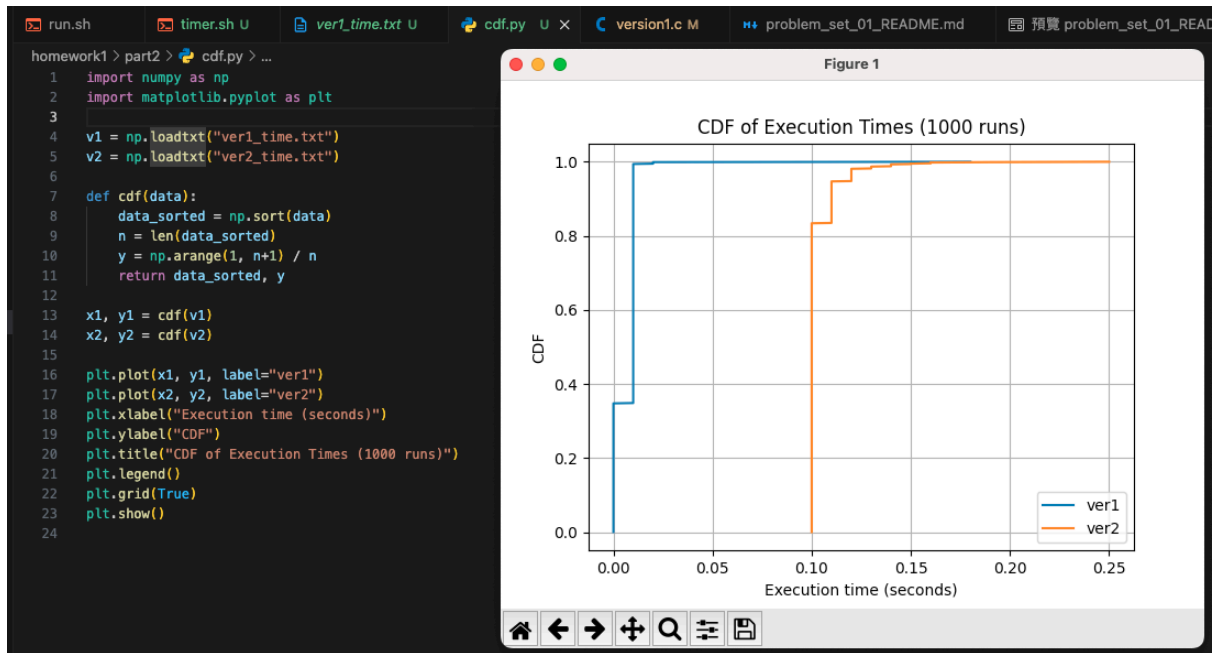
fprintf is a library call that buffers the output, with the buffer size usually 4KB or 8KB, and it won't actually print out until fclose/flush or the buffer is full. So, although f1 has 30000 characters, it actually has around 80 system calls, compared to f2, which uses write, which is a user -> kernel system call every time.

So, although it uses pthread_create, we can't guarantee that f1 will always print out faster than f2. But since the system call difference is huge, it is expected that f1 will occur before f2 almost every time.

Part 2: Understanding the differences

- Task 1: I wrote a [timer.sh](#) to print out all the execution times in ver1_timer.txt and ver2_time.txt. And a Python script that can read the execution time and plot the CDF with matplotlib.

- Task 2:



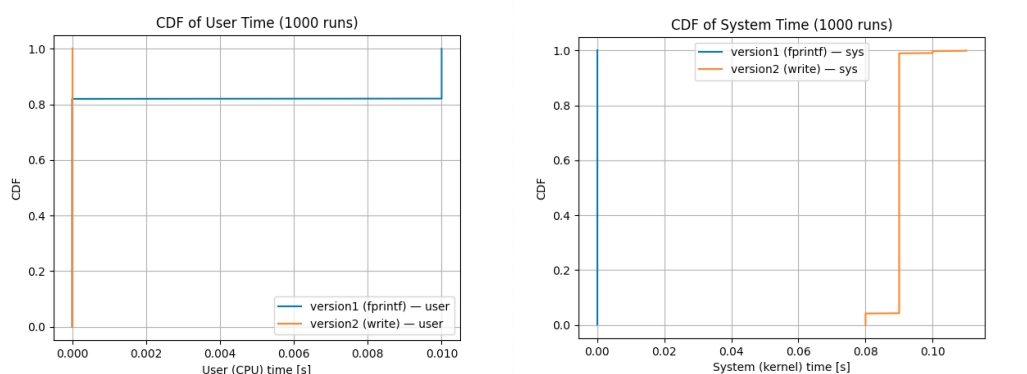
The underlying reason of why the plot looks like this is:

`ver1` (the `fprintf` version): The CDF will be farther left (shorter execution times) and tighter (less spread). Almost all runs finish quickly.

`ver2` (the write version): The CDF will be shifted right (longer execution times) and maybe slightly more variable.

That's because `ver2` performs 100,000 syscalls instead of ~70. As stated in Q1, the reason behind the graph looks like this is because `fprintf` has user-space buffering, so fewer system calls are compared to system call I/O(`ver2`). Syscalls dominate the runtime difference, so `ver2` is much slower and more variable depending on system load.

- Task 3:



I first wrote a script that records version 1 and version 2's user time and system time in the `v1_raw` and `v2_raw` files. After that, I use a python script to separate the user time and system time into two different graphs. According to the result, we can find that since library calls use a buffer, most of the time spent is in user time, with formatting and buffering.

As for system time, according to our observation, the time consumed by a system call (write) is significantly more than that of a library call. Since library call is ~70 with buffer, and the system call is 10000 times.

Overall takeaway

1. version 1 (library call):
 - More user time, negligible system time.
 - Much faster overall.
2. version2 (system call):
 - Less user time, but system time dominates.
 - Much slower overall.
3. The plots will show two clearly separated CDFs:
 - For user time → ver1 slightly to the right of ver2.
 - For system time → ver2 way to the right of ver1.

- Task 4:

Do I see same execution time across runs?

No.

Even when running the *same program* repeatedly, the measured times will fluctuate slightly.

How to make runs more consistent

1. Control system load
 - Close other applications.
 - Run on an idle machine.
2. Fix CPU frequency
 - Disable dynamic frequency scaling (Turbo Boost, power-saving).
 - Prevents the CPU from changing speed during runs.
3. Pin to a single CPU core
 - Use taskset or sched_setaffinity so your process always runs on the same core.
 - Avoids migration overhead and cache flushes.
4. Set a higher priority
 - Use real-time scheduling (chrt) so the OS is less likely to preempt your process.
 - Reduces variability from other processes.

5. Discard warm-up runs

- First runs may load code/data into cache and page cache → slower.
- Ignore the first few runs and measure the steady state.